

TITANIC CLASSIFICATION – PREDICTING SURVIVAL



Table of Contents

<u>IMPLEMENTATION</u>	1
1)DATA CLEANING AND FEATURE ENGINEERING:	1
2) DATA SPLITTING.....	2
3)MODELS SELECTED	2
4) RESULTS AND ANALYSIS.....	4
<u>SAMPLE RUN:</u>	5
<u>SOURCE CODE:</u>	8
<u>REFERENCES:</u>	12

Note: The report is longer than 2-3 pages due to the multiple implementation details I performed(I used an extra model so I need an extra page) and document formatting(The sample run and code has several pages).

Implementation

This assignment focused on machine learning, more specifically, it focused on how to train certain data models to make predictions. I used 3 models, each having different results, to predict the survival rate from the given titanic dataset. I first cleaned and split the data, applied feature engineering, trained my models with some tuning, and analyzed the results of my models by using several different metrics. Below are the implementation steps in details.

1)Data cleaning and Feature Engineering:

I first loaded the train.csv dataset from the Kaggle website into my coding environment, handling errors that may occur with loading files.

Next, I cleaned the loaded data and attempted feature engineering:

- 1- The “**PassengerID**”, “**Name**”, and “**Ticket**” columns are dropped as they provide no useful information in predicting survival.
- 2- The “**Cabin**” column is dropped as it has multiple missing values.

- 3- The missing values in the “**Embarked**” column are filled with the most common value.(This is done by using the mode() operation).
- 4- The “**Embarked**” column is then converted into multiple one-hot encoded columns where each unique value is separated into its own binary columns.
- 5- The missing values in the “**Fare**” column are filled with the median value using the median() operation.(I used median so it will not get affected by high values which are uncommon).
- 6- The “**Sex**” column is converted into a numeric field, 0 for male and 1 for female. This transformation is necessary so that the models will be able to make predictions more efficiently.
- 7- I identified missing values in the “**Age**” column by using the KNN model, where it looks at the 7 closest neighbors to estimate the missing values. The reason I selected this method is to find new ways to predict an attribute value and to explore as many models and machine learning techniques as I could.
- 8- Lastly, I created a “**FamilySize**” column that represents the total number of family members in each passengers’ family, including themselves. This feature is significant as it greatly affects the prediction due to the fact that very large and very small families had low survival rates, while medium families had a higher chance of survival.

2) Data Splitting

After cleaning the data, I removed the “**Survived**” column from the input data, because it is the label the model needs to predict, and saved it in the output data so it can be used to check if the model correctly predicted survival rate. Next, the training data is split into **train** and **validation** datasets, with 80% of the data is used for training and 20% is used for testing/validation.

3)Models Selected

In this part of the assignment, we were required to select at least two algorithms and train models using the provided training data. I trained three different models which are: random forest, gradient boosting, and logistic regression.

1- Random Forest:

The first model I selected was Random Forest. One of the main reasons I selected it was due to the fact it had an interesting name, which encouraged me to research more about it. Random Forest is a very powerful model that has a high performance. It works by building multiple decision trees during training and making predictions based on them.

To train the rainforest model, I first had to create the model and manually tunned it for better results:

```
model1=RandomForestClassifier(n_estimators=600,max_depth=7,min_samples_split=5,random_state=1)
```

1- **n_estimators= 600:**

Indicates the number of trees in the model. (The more trees the better however it could lead to slower time). I first tried 200 however I got poor results, after increasing it to 600, the results significantly improved.

2- **max_depth=7:**

Indicates the maximum depth of the tree and prevents overfitting.

I initially tried setting the max_depth=5 however I obtained better results when I changed max_depth to 7.

3- **min_samples_split=5:**

Indicates the minimum number of samples to split a node so it can prevent any useless splits leading to overfitting.

I initially tried setting the min_samples_split=10 however I obtained better results when I changed min_samples_split to 5. This is because the model will be allowed to capture smaller patterns in the data, making more accurate predictions.

4- **random_state=1:**

Controls the randomness of the model so we get the same results each time.

2- **Gradient Boosting:**

The second model I selected was Gradient Boosting. One of the main reasons I selected it was due to its high performance. It is highly accurate and powerful due to that fact it builds several decision trees, where each one of them corrects the errors of the previous trees. (This model detects more actual survivors and has less false negatives).

To train the Gradient Boosting model, I first had to create the model and manually tuned it for better results:

```
model2= GradientBoostingClassifier(n_estimators=600, learning_rate=0.1,
max_depth=7, random_state=1)
```

1- **n_estimators= 600:**

Indicates the number of trees in the model. (The more trees the better however it could lead to slower time). Similar to the Random Forest model, I first tried 200 however I got poor results, after increasing it to 600, the results significantly improved.

2-**learning rate=0.1:**

The learning rate controls how much each model learns at each step, making improvement from the previous model, which is one of the key ideas of the Gradient Boosting model. I selected 0.1 as the learning rate. Although it is small leading to a slower learning, it gives more accurate results.

3- **max_depth=7:**

Indicates the maximum depth of the tree and prevents overfitting.

4- **random_state=1:**

Controls the randomness of the model so we get the same results each time.

3- **Logistic Regression**

The last model I used was Logistic Regression. The main reason I selected it is due to the fact it is commonly used. Additionally, I wanted to explore and see how it compared to the Gradient Boosting and Random Forest models. After further research, Logistic Regression is used to

predict binary outcomes, therefore it could be used to predict whether the passenger survived (survived =1) or died (died=0).

```
model3= LogisticRegression(C=0.5, max_iter=600, random_state=1)
```

Similarly to the previous models, I first had to create the linear regression model and manually tuned it for better results:

1- C=0.5:

The C value is used to control regularization (technique to reduce overfitting). Having a high C value will cause weak regularization, while having a low C value will cause a strong regularization. I chose 0.5 to maintain balance.

2- Max_iter=600:

Max_iter represents the number of times the model will iterate over the data to train. Although having a high value ensures training the model efficiently, it will take a longer amount of time.

3- random_state=1:

Controls the randomness of the model so we get the same results each time.

After creating each model, I trained them and evaluated the models using different metrics. I then printed the results and created a corresponding correlation heatmap to better visualize the results.

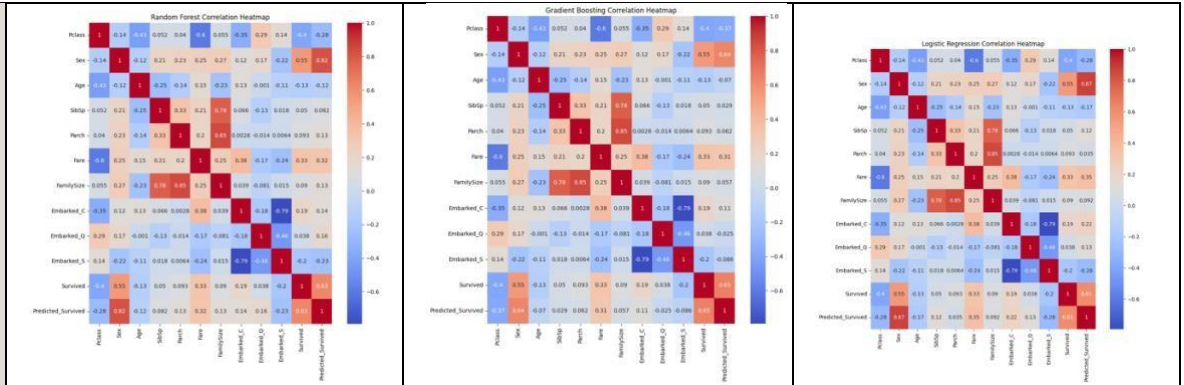
4) Results and Analysis

To evaluate my models, I used a several different evaluation metrics. The metrics I used are **accuracy**, **precision**, **recall**, and **F1 score**. Additionally, for each model, I provided two visual aids, the **confusion matrix** and **correlation heatmap**. The confusion matrix compares the predicted values against the actual values, while the correlation heatmap shows the correlation/relationships between variables using a color-coded matrix.

After running the code and testing my models, I got different outputs for each model:

Metrics	Random Forest	Gradient Boosting	Logistic Regression
Accuracy	0.8212	0.8324	0.8101
Precision	0.8387	0.8144	0.7857
Recall	0.7027	0.7703	0.7432
F1 Score	0.7647	0.7917	0.7639
Confusion Matrix	[[95 10] [22 52]]	[[92 13] [17 57]]	[[90 15] [19 55]]

Correlation Heatmap



From the table above, it is clear that out of the three models, the model with the best overall results is the Gradient Boosting model. Although it has a lower precision than the Random Forest model, it has higher accuracy, recall and f1 score indicating that the model got multiple predictions correct and has few false negatives. The second model with the best results is the Random Forest model with the highest precision out of the three indicating it has the fewest false positives(false positives is when the model predicts survival but the passenger did not actually survive). However, the Random Forest model has the lowest recall value indicating it missed multiple survivals (many false negatives).Lastly, the logistic regression model is in third place having a balance of precision and recall. Although it has a lower performance compared to the other models, it is simple with a good accuracy score.

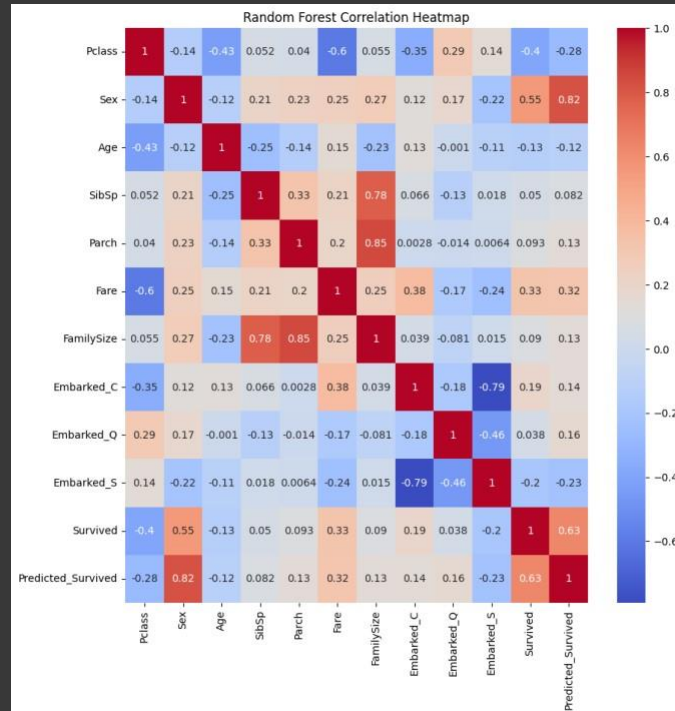
Sample Run:

----Random Forest Algorithm Results----

Accuracy: 0.8212
Precision: 0.8387
Recall: 0.7027
F1 Score: 0.7647

Confusion Matrix:

```
[[95 10]
 [22 52]]
```

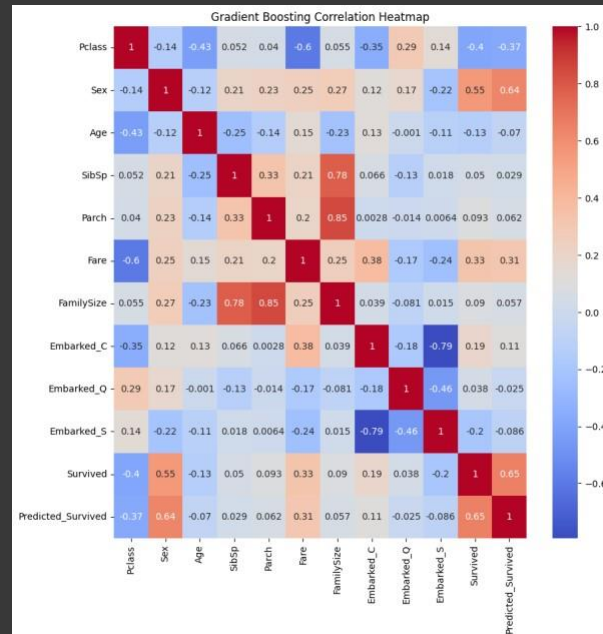


----Gradient Boosting Algorithm Results---

Accuracy: 0.8324
Precision: 0.8143
Recall: 0.7703
F1 Score: 0.7917

Confusion Matrix:

```
[[92 13]
 [17 57]]
```

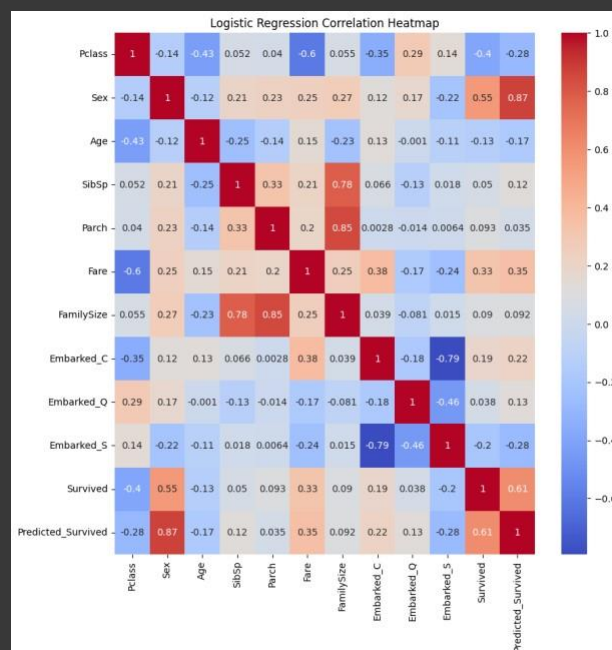


----Logistic Regression Algorithm Results---

Accuracy: 0.8101
Precision: 0.7857
Recall: 0.7432
F1 Score: 0.7639

Confusion Matrix:

```
[[90 15]
 [19 55]]
```



Source Code:

```
import pandas as pd #for data processing from sklearn.impute import
KNNImputer #use it to fill missing values from sklearn.model_selection
import train_test_split #to split the data for training and validation
from sklearn.ensemble import
RandomForestClassifier,GradientBoostingClassifier #for the first model
(Random Forest) and the second model (Gradient Boosting) from
sklearn.linear_model import LogisticRegression #for the third model
(Logistic Regression) from sklearn.metrics import
accuracy_score,precision_score,recall_score,f1_score,
confusion_matrix
#for evaluating the model import numpy as np
#for the heatmap import matplotlib.pyplot as plt
#for the heatmap import seaborn as sns #for the
heatmap

def cleanData(train_data):
    train_data= train_data.drop(['PassengerId','Name','Ticket','Cabin'],
axis=1) #removing irrelevant data- (not useful to predict survival) (axis=1
means the operation is applied to columns)
train_data["Embarked"].fillna(train_data["Embarked"].mode()[0]) #fill the
missing values in the "Embarked" column with the most common value(using
mode()) train_data["Fare"].fillna(train_data["Fare"].median()) #fill the
missing values in the "Fare" column with the median value (the middle
value so it will not get affect by very large numbers which is uncommon)
train_data["Sex"]=train_data["Sex"].map({"male":0,"female":1}) #converts
sex into a numeric feild, 0 for male and 1 for female
train_data["FamilySize"]= train_data["SibSp"]+train_data["Parch"]+1 #
create family size column and added 1 for the passanger itself
impute_attributes=
["Pclass","Sex","SibSp","Parch","Fare","Age","FamilySize"] #the attributes
that will make descisions off of to decide the suitable age value
KNNImpute=KNNImputer(n_neighbors=7) # will look at the seven nearest neighbors
and make descisions based on them

train_data[impute_attributes]=KNNImpute.fit_transform(train_data[impute_at
tributes]) train_data["Age"]=train_data["Age"].astype(int) #converts age
back to int because KNN converts it to float
```



```

train_data= pd.get_dummies(train_data, columns=["Embarked"])
#converts embarked into multiple one-hot encoded columns so each
category is independent of the other
train_data["Pclass"]=train_data["Pclass"].astype(int) #converts Pclass
back to int because KNN converts it to float and we want it numeric
return train_data

def
main():

try:
    train_data= pd.read_csv("train.csv")# Reading the file
except FileNotFoundError:
    print("File not found. Please check the file
path.")    exit()    try:
    train_data= cleanData(train_data) #cleaning the data
    input_data=train_data.drop(columns=["Survived"]) #dropping the
target so the model can learn.
    output_data= train_data["Survived"] #The target
    #performing the splitting:
train_input,validation_input,train_output,validation_output=
train_test_split(input_data,output_data, test_size=0.2,random_state=42)
    #Modell: Random forest
    #Step1- creating the model:

modell=RandomForestClassifier(n_estimators=600,max_depth=7,min_samples_spl
it=5,random_state=1)
    #Step2- Training the model:
    modell.fit(train_input,train_output)    #Step3-
predict on validation data    modell_prediction=
modell.predict(validation_input)
    #Step4- evaluate the model    modell_accuracy=
accuracy_score(validation_output,modell_prediction)    modell_precision=
precision_score(validation_output,modell_prediction)    modell_recall=
recall_score(validation_output,modell_prediction)    modell_f1=
f1_score(validation_output,modell_prediction)
    modell_confusion_matrix=
confusion_matrix(validation_output,modell_prediction)
    #Step5- print results for modell    print("----
Random Forest Algorithm Results----")    print()

```



```

    print(f" Accuracy: {modell1_accuracy:.4f}")
print(f" Precision: {modell1_precision:.4f}")
print(f" Recall: {modell1_recall:.4f}")
print(f" F1 Score: {modell1_f1:.4f}")
print()    print(" Confusion Matrix: \n ")
print(modell1_confusion_matrix)    print()
    #step6- create correlation heat map
    Modell1DF= validation_input.copy()
    Modell1DF["Survived"]=validation_output #The actual survival results
    Modell1DF["Predicted_Survived"]=modell1_prediction#The predicted survival
    results    modell1_correlation_matrix= Modell1DF.corr()
plt.figure(figsize=(10,10))
sns.heatmap(modell1_correlation_matrix,annot=True, cmap="coolwarm")
plt.title("Random Forest Correlation Heatmap")    plt.show()    print()

    #Model2: Gradient Boosting    #Step1- creating the model:    model2=
GradientBoostingClassifier(n_estimators=600, learning_rate=0.1,
max_depth=7, random_state=1)    #Step2- Training the model:
    model2.fit(train_input,train_output)    #Step3-
predict on validation data    model2_prediction=
model2.predict(validation_input)
    #Step4- evaluate the model    model2_accuracy=
accuracy_score(validation_output,model2_prediction)    model2_precision=
precision_score(validation_output,model2_prediction)    model2_recall=
recall_score(validation_output,model2_prediction)    model2_f1=
f1_score(validation_output,model2_prediction)
model2_confusion_matrix=
confusion_matrix(validation_output,model2_prediction)
    #Step5- print results for model2    print("----
Gradient Boosting Algorithm Results---")    print()
print(f" Accuracy: {model2_accuracy:.4f}")
print(f" Precision: {model2_precision:.4f}")
print(f" Recall: {model2_recall:.4f}")    print(f" F1
Score: {model2_f1:.4f}")    print()
    print(" Confusion Matrix: \n ")

```



```

    print(model2_confusion_matrix)
print()
    #step6- create correlation heat map
    Model2DF= validation_input.copy()
    Model2DF["Survived"]=validation_output #The actual survival results
    Model2DF["Predicted_Survived"]=model2_prediction#The predicted survival
    results    model2_correlation_matrix= Model2DF.corr()
plt.figure(figsize=(10,10))
sns.heatmap(model2_correlation_matrix,annot=True, cmap="coolwarm")
plt.title("Gradient Boosting Correlation Heatmap")    plt.show()
print()

    #model3: Logistic Regression
    #step1- creating the model:
    model3= LogisticRegression(C=0.5, max_iter=600, random_state=1)
    #step2- Training the model:
    model3.fit(train_input,train_output)    #step3-
    predict on validation data    model3_prediction=
    model3.predict(validation_input)
    #step4- evaluate the model    model3_accuracy=
    accuracy_score(validation_output,model3_prediction)    model3_precision=
    precision_score(validation_output,model3_prediction)    model3_recall=
    recall_score(validation_output,model3_prediction)    model3_f1=
    f1_score(validation_output,model3_prediction)
    model3_confusion_matrix=
    confusion_matrix(validation_output,model3_prediction)
    #step5- print results for model3    print("----
    Logistic Regression Algorithm Results---")    print()
    print(f" Accuracy: {model3_accuracy:.4f}")    print(f"
    Precision: {model3_precision:.4f}")    print(f" Recall:
    {model3_recall:.4f}")    print(f" F1 Score:
    {model3_f1:.4f}")    print()    print(" Confusion
    Matrix: \n ")    print(model3_confusion_matrix)
print()
    #step6- create correlation heat map
    Model3DF= validation_input.copy()
    Model3DF["Survived"]=validation_output #The actual survival results
    Model3DF["Predicted_Survived"]=model3_prediction#The predicted survival
    results

```



```

    model3_correlation_matrix= Model3DF.corr()
plt.figure(figsize=(10,10))
sns.heatmap(model3_correlation_matrix,annot=True, cmap="coolwarm")
plt.title("Logistic Regression Correlation Heatmap")    plt.show()
print()
    except Exception as
e:
    print(f"An error occurred: {e}")

if __name__ == '__main__':
    main()

```

Note: After running the code in both the terminal and google colab, I got different results(minor differences) likely due to differences in library versions. The results in this document is based on google colab.

References:

For this assignment, I watched several YouTube videos and read websites that helped me understand how to train the models and code them. Below are some of the references I used:

- [1] A. Ali, "How I Solved The Kaggle's Titanic Machine Learning From Disaster Competition?," *Medium*, May 06, 2023. <https://medium.com/@asaadaali/how-i-solved-thekaggles-titanic-machine-learning-from-disaster-competition-1d24f998059f> (accessed Apr. 23, 2025).
- [2] "Titanic - Machine Learning from Disaster," *kaggle.com*. <https://www.kaggle.com/competitions/titanic/overview>
- [3] M. Gupta, "ML | Linear Regression - GeeksforGeeks," *GeeksforGeeks*, Sep. 13, 2018. <https://www.geeksforgeeks.org/ml-linear-regression/>

